Practical 6

Solve a constraint satisfaction problem such as map coloring, Sudoku or N-Queens using backtracking and consistency checking.

Objective

To model the N-Queens problem as a Constraint Satisfaction Problem (CSP) and solve it using backtracking search combined with forward-checking (consistency checking) to prune the search space.

Procedure

1. Model N-Queens as a CSP: one variable per row, its domain is the set of columns {0 ... N-1}, with the constraint that no two queens share a column or diagonal.
2. Implement is_consistent() to check row-by-row that a candidate column does not clash in column or diagonal with any already-placed queen.
3. Implement forward_check() to remove, from the domains of not-yet-assigned rows, any column value that would immediately conflict with the newly placed queen (arc-consistency-style pruning).
4. Implement the recursive backtrack() search: try each remaining value in the current row's domain, apply forward checking, recurse to the next row, and undo (restore domains) on failure.
5. Run the solver for an 8x8 board (N = 8) and print the resulting board along with the row->column placement of each queen.
6. Record the number of backtracks performed and the total number of consistency checks made during the search.

Program / Code

```
# ================================================================
# PRACTICAL 6
# Constraint Satisfaction Problem - N-Queens
# Backtracking Search with Forward Checking (Consistency Checking)
# ================================================================

N = 8

backtrack_count = 0
consistency_checks = 0


def is_consistent(row, col, placement):
    """Check row/col/diagonal consistency against already-placed queens."""
    global consistency_checks

    for r, c in enumerate(placement[:row]):
        consistency_checks += 1
        if c is None:
```

```
            continue
        if c == col:                        # same column
            return False
        if abs(c - col) == abs(r - row):    # same diagonal
            return False
    return True


def forward_check(domains, row, col, n):
    """Remove col / diagonal values from domains of rows below `row`.
    Returns the removed values so they can be restored on backtrack,
    or None if any domain becomes empty (failure)."""
    removed = {}
    for r in range(row + 1, n):
        to_remove = set()
        for c in domains[r]:
            if c == col or abs(c - col) == (r - row):
                to_remove.add(c)
        if to_remove:
            removed[r] = to_remove
            domains[r] -= to_remove
            if not domains[r]:
                return removed, False
    return removed, True


def backtrack(row, placement, domains, n):
    global backtrack_count

    if row == n:
        return True

    for col in sorted(domains[row]):
        if is_consistent(row, col, placement):
            placement[row] = col

            removed, ok = forward_check(domains, row, col, n)

            if ok and backtrack(row + 1, placement, domains, n):
                return True

            # Undo forward checking (restore domains)
            for r, vals in removed.items():
                domains[r] |= vals

            placement[row] = None
            backtrack_count += 1

    return False


def solve_n_queens(n):
    placement = [None] * n
    domains = [set(range(n)) for _ in range(n)]
    success = backtrack(0, placement, domains, n)
    return placement if success else None
```

```
def print_board(placement, n):
    for r in range(n):
        row_cells = []
        for c in range(n):
            row_cells.append("Q" if placement[r] == c else ".")
        print(" ".join(row_cells))


if __name__ == "__main__":
    print("=" * 55)
    print(f"     N-QUEENS CSP  (N = {N})")
    print("     Backtracking + Forward Checking")
    print("=" * 55)

    solution = solve_n_queens(N)

    if solution:
        print("\nSolution Found:\n")
        print_board(solution, N)
        print("\nQueen positions (row -> column):")
        for r, c in enumerate(solution):
            print(f"  Row {r} -> Column {c}")
    else:
        print("No solution found.")

    print("-" * 55)
    print(f"Backtracks performed     : {backtrack_count}")
    print(f"Consistency checks made  : {consistency_checks}")
    print("=" * 55)
```

Output

```
=======================================================
     N-QUEENS CSP  (N = 8)
     Backtracking + Forward Checking
=======================================================

Solution Found:

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Queen positions (row -> column):
  Row 0 -> Column 0
  Row 1 -> Column 4
  Row 2 -> Column 7
  Row 3 -> Column 5
```

```
  Row 4 -> Column 2
  Row 5 -> Column 6
  Row 6 -> Column 1
  Row 7 -> Column 3
----------------------------------------------------
Backtracks performed     : 80
Consistency checks made  : 325
====================================================
```

Conclusion

The 8-Queens CSP was solved using backtracking search with forward checking. The solver placed all 8 queens with no two attacking each other, needing only 80 backtracks and 325 consistency checks to find a valid configuration — far fewer than the 8^8 (about 16.7 million) naive placements that would be tried without any pruning. This confirms that combining constraint propagation (forward checking, which proactively shrinks the domains of future variables) with backtracking search dramatically reduces the effective search space for CSPs, allowing an exponential problem like N-Queens to be solved almost instantly for practical values of N.